
TRABAJO FINAL DE UNIDAD — UT2

Tácticas de Arquitectura

Requerimientos no-funcionales y justificación de las tácticas de arquitectura seleccionadas, con su implementación verificable en una API REST desplegada en Docker.

Rendimiento · Mantener múltiples copias del cómputo

Seguridad · Autenticar actores

Seguridad · Limitar el acceso

AUTORES

Adolfo Bravo — Agustin Cigaran — Brahina Nuñez - Matias Perez

ENTREGA

Parte 1 — Requerimientos y tácticas

Table of Contents

1. Combinación de tácticas seleccionada	2
2. Requerimientos no-funcionales	3
3. Cómo las tácticas seleccionadas satisfacen los requerimientos	8
4. Interacción entre las tácticas (trade-offs)	10
5. Verificación	11
6. Fuentes	12

1. Combinación de tácticas seleccionada

De las combinaciones propuestas en la consigna se eligió la primera:

Mantener múltiples copias de cómputo para rendimiento, y dos tácticas de la categoría “resistir ataques” para seguridad.

En el catálogo de la Unidad 2, la primera táctica figura como «mantener múltiples copias del cómputo» y pertenece a la rama «gestionar los recursos» del catálogo de rendimiento (la otra rama es «controlar la demanda de recursos»). Las dos tácticas de seguridad pertenecen a la rama «resistir ataques», la más numerosa de las cuatro del catálogo de seguridad (detectar, resistir, reaccionar, recuperarse de ataques).

T1	Rendimiento	Gestionar los recursos	Mantener múltiples copias del cómputo	<code>docker-compose.yaml</code> (3 réplicas) + <code>gateway/nginx.conf</code> (upstream con <code>least_conn</code>)
T2	Seguridad	Resistir ataques	Autenticar actores	<code>api/src/auth.js</code> (scrypt + JWT HS256) y <code>api/src/server.js</code> (middleware <code>autenticar</code>)
T3	Seguridad	Resistir ataques	Limitar el acceso	<code>gateway/nginx.conf</code> (rate limiting, verbos permitidos, único punto de entrada) + <code>api/src/server.js</code> (bloqueo de cuenta, control por rol) + red Docker <code>internal</code>

Nota sobre T1. Para que “mantener múltiples copias del cómputo” sea aplicable, el servicio tuvo que diseñarse **sin estado en memoria compartida**: todo el estado que debe ser visto por igual desde cualquier réplica (contadores de carga, contador de intentos fallidos de login) se externalizó a Redis. Esta decisión es parte de la táctica, no un accesorio: sin ella, dos peticiones consecutivas del mismo cliente atendidas por réplicas distintas darían resultados incoherentes.

1.1 Tácticas complementarias empleadas

T1, T2 y T3 son las tres tácticas exigidas por la consigna, pero la implementación apoya varias decisiones en otras tácticas de los mismos catálogos. Se nombran acá con el vocabulario del catálogo para no confundirlas con las tres principales:

Calendarizar recursos	Rendimiento · gestionar los recursos	Política <code>least_conn</code> del <code>upstream</code> : asigna recursos a eventos según un criterio (el catálogo da <i>first-in/first-out</i> y <i>round robin</i> como ejemplos)
Autorizar actores	Seguridad · resistir ataques	Control por rol sobre las operaciones de escritura (RNF-07)
Separar entidades	Seguridad · resistir ataques	Aislamiento en contenedores y red Docker <code>internal</code>
Limitar la exposición	Seguridad · resistir ataques	<code>server_tokens off</code> , superficie HTTP acotada al contrato (RNF-06)
Restringir el ingreso	Seguridad · reaccionar a ataques	Bloqueo de cuenta tras varios intentos fallidos de autenticación (RNF-05)
Replicación	Disponibilidad · redundancia	Réplicas intercambiables que sostienen RNF-03
Empaquetar también las dependencias	Facilidad de despliegue · sistema desplegado	Imagen única <code>ut2-tfu/api:1.0.0</code> con sus dependencias
Scripts para la implementación	Facilidad de despliegue · pipeline	<code>docker-compose.yaml</code> y los scripts de <code>scripts/</code>
Binding en tiempo de configuración	Facilidad de modificación · diferir el binding	Parámetros por entorno (<code>.env</code>) leídos al iniciar la ejecución (RNF-02)

Vale la pena marcar una distinción del catálogo de seguridad: el **rate limiting del gateway** es *limitar el acceso* (resistir ataques), mientras que el **bloqueo de cuenta** es, en rigor, *restringir el ingreso*, que el catálogo ubica en la rama **reaccionar a ataques** — su ejemplo literal es “bloquear a un usuario tras varios intentos fallidos de autenticación”. Se documentan juntos en T3 porque operan sobre el mismo requerimiento (RNF-05), pero pertenecen a ramas distintas.

2. Requerimientos no-funcionales

Se describen a continuación los requerimientos que las tácticas seleccionadas permiten satisfacer. Cada uno se presenta como un **escenario de atributo de calidad** (fuente, estímulo, artefacto, entorno, respuesta y medida de respuesta), que es la forma de expresar un requerimiento no-funcional de manera verificable.

El encuadre es el de los catálogos de la Unidad 2. Las tácticas de rendimiento buscan **generar una respuesta a los eventos que llegan al sistema dentro de una restricción basada en tiempo o en recursos**, y se dividen en controlar la demanda de recursos o gestionar los recursos que responden. Las de seguridad se organizan por la analogía de una instalación física —muros, cámaras, alarmas y respaldos— en cuatro ramas: detectar, resistir, reaccionar y recuperarse de los ataques.

RNF-01 — Rendimiento bajo carga concurrente

Atributo	Rendimiento (<i>performance</i>)
Fuente del estímulo	Un conjunto de clientes externos de la API (curl / Postman / otro sistema).
Estímulo	Llegan 60 peticiones concurrentes al recurso <code>GET /api/report</code> , que ejecuta un cálculo intensivo en CPU.
Artefacto	El servicio de aplicación completo (gateway + réplicas).
Entorno	Operación normal, todas las réplicas sanas, capacidad de CPU acotada por réplica (0,5 vCPU).
Respuesta	El sistema atiende la totalidad de las peticiones sin rechazar ninguna ni degradarse a errores.
Medida de respuesta	El lote completo se resuelve en menos de un tercio del tiempo que insume la misma carga sobre una sola instancia; el <i>throughput</i> agregado supera las 10 req/s frente a las ~4 req/s de una instancia única.

Descripción en prosa. El sistema debe sostener ráfagas de peticiones costosas en CPU sin que el tiempo de respuesta percibido crezca linealmente con la concurrencia. Como el trabajo de cada petición no puede acelerarse (es CPU-bound e indivisible), la única forma de mejorar el tiempo del lote es atender varias peticiones **en paralelo**.

RNF-02 — Escalabilidad horizontal sin modificar la aplicación

Atributo	Facilidad de modificación — escalabilidad , una de las categorías con nombre propio de este atributo (junto con variabilidad, portabilidad e independencia de la ubicación)
Fuente del estímulo	El equipo de operaciones ante un aumento sostenido de la demanda.
Estímulo	Se solicita aumentar la capacidad de cómputo del servicio.
Artefacto	Configuración de despliegue (<code>docker-compose.yaml</code> , <code>nginx.conf</code>).
Entorno	Sistema en producción.
Respuesta	Se agrega o quita capacidad sin tocar el código fuente ni recompilar la imagen , y sin que los clientes deban

	cambiar la URL a la que apuntan: la decisión de cuántas copias hay se difiere al tiempo de configuración .
Medida de respuesta	Agregar una réplica requiere modificar solo la configuración de despliegue ; el cambio se aplica en menos de 1 minuto y 0 líneas de código de negocio se ven afectadas.

RNF-03 — Continuidad del servicio ante la caída de una réplica

Atributo	Disponibilidad (beneficio derivado de T1)
Fuente del estímulo	Falla interna: un contenedor de la aplicación se detiene o deja de responder.
Estímulo	Caen 2 de las 3 réplicas.
Artefacto	Gateway y pool de réplicas.
Entorno	Operación degradada.
Respuesta	El gateway detecta la réplica no disponible, la excluye del pool y reencamina las peticiones a las réplicas sanas. Las peticiones en vuelo hacia la réplica caída se reintentan sobre otra.
Medida de respuesta	0 % de peticiones fallidas durante la degradación; los clientes no observan errores 5xx.

Este requerimiento no era exigido por la combinación elegida, pero surge naturalmente de mantener múltiples copias del cómputo y se documenta porque es una de las razones por las que la táctica se elige en la práctica.

RNF-04 — Solo actores autenticados acceden a los recursos de negocio

Atributo	Seguridad (confidencialidad e integridad)
Fuente del estímulo	Un atacante no autenticado, o un usuario legítimo con un token adulterado o vencido.
Estímulo	Intenta invocar <code>GET /api/products</code> , <code>POST /api/products</code> o <code>GET /api/report</code> sin credenciales válidas.

Artefacto	Todos los recursos bajo el prefijo <code>/api/</code> .
Entorno	Operación normal.
Respuesta	El sistema rechaza la petición antes de ejecutar cualquier lógica de negocio y responde <code>401 Unauthorized</code> indicando el motivo, sin filtrar información sensible.
Medida de respuesta	100 % de las peticiones sin token válido son rechazadas. Ningún token firmado con una clave distinta, con la firma alterada o con <code>exp</code> vencido es aceptado. Ninguna contraseña se almacena ni se transmite en texto plano.

RNF-05 — Resistencia a ataques de fuerza bruta sobre credenciales

Atributo	Seguridad
Fuente del estímulo	Un atacante automatizado desde una misma dirección de origen.
Estímulo	Envía un volumen alto de intentos de autenticación contra <code>POST /auth/login</code> probando contraseñas.
Artefacto	Endpoint de autenticación (gateway + servicio de aplicación).
Entorno	Operación normal; el resto de los usuarios sigue operando.
Respuesta	El sistema aplica dos frenos independientes y en capas distintas : (a) el gateway limita la tasa de peticiones por IP y absorbe el exceso en el borde; (b) la aplicación restringe el ingreso de la cuenta atacada tras N fallos y deja de evaluar la contraseña.
Medida de respuesta	Tras 5 intentos fallidos en 60 s la cuenta queda bloqueada por 60 s. Superadas las 60 peticiones/minuto por IP en <code>/auth/login</code> , el gateway responde <code>429</code> sin consumir recursos de las réplicas . El costo efectivo del ataque pasa de miles de intentos por minuto a menos de 5.

RNF-06 — Superficie de ataque mínima

Atributo	Seguridad
Fuente del estímulo	Un atacante con acceso a la red donde está desplegado el sistema.
Estímulo	Intenta conectarse directamente a las instancias de la aplicación, a la base de estado (Redis), o usar verbos HTTP no previstos por la API.
Artefacto	Topología de red del despliegue y configuración del gateway.
Entorno	Operación normal.
Respuesta	Los componentes internos no son alcanzables: están separados en contenedores propios, no publican puertos al host y residen en una red sin ruta al exterior. Los verbos HTTP fuera del contrato son rechazados en el borde. El gateway limita la exposición : no revela su versión.
Medida de respuesta	1 solo puerto expuesto en todo el sistema (el del gateway). Conexiones directas a los puertos 3000 y 6379 desde el host: sin respuesta . Verbos no previstos: 405 . Cabecera Server sin número de versión.

RNF-07 — Autorización diferenciada por rol

Atributo	Seguridad
Fuente del estímulo	Un usuario correctamente autenticado pero sin privilegios suficientes.
Estímulo	Un usuario con rol user intenta ejecutar POST /api/products (operación de escritura reservada a administradores).
Artefacto	Recursos de escritura de la API.
Entorno	Operación normal.
Respuesta	La táctica de autorizar actores determina qué puede hacer un actor ya autenticado —acá, por rol— y la petición se rechaza con 403 Forbidden ; la operación no se ejecuta ni deja rastro en el estado del sistema.

Medida de respuesta	100 % de los intentos de escritura con rol insuficiente son rechazados; el mismo recurso responde <code>201</code> para un actor con rol <code>admin</code> .

3. Cómo las tácticas seleccionadas satisfacen los requerimientos

T1 — Mantener múltiples copias del cómputo → RNF-01, RNF-02, RNF-03

En qué consiste la táctica. El catálogo la define como una táctica de la rama **gestionar los recursos**, cuyo propósito es **reducir la contención** que produce tener todo el procesamiento concentrado en un solo componente; sus ejemplos son los servicios replicados en microservicios y el *pool* de servidores web. Es decir: en lugar de intentar reducir el trabajo que demanda cada petición —lo que corresponde a la otra rama, controlar la demanda de recursos—, se **aumenta la cantidad de unidades de cómputo capaces de atenderla**, se replica el servicio y se coloca un intermediario que reparte la carga entre las copias. Cada copia es funcionalmente idéntica e intercambiable.

Cómo se implementó.

1. **Tres réplicas idénticas** (`api1`, `api2`, `api3`) construidas a partir de **una única imagen** (`ut2-tfu/api:1.0.0`). Que compartan artefacto no es un detalle: garantiza que las copias son verdaderamente equivalentes y que cualquiera puede atender cualquier petición.
2. **Servicio sin estado en memoria.** El estado compartido vive en Redis (contadores de reparto de carga, contador de intentos fallidos). El JWT es *self-contained* y se valida con la clave secreta, por lo que **no hace falta sesión en el servidor**: cualquier réplica puede validar un token emitido por otra. Esto es lo que hace que las copias sean intercambiables.
3. **Balanceador de carga** (nginx) con política `least_conn`. Elegir la política es aplicar la táctica de **calendarizar recursos** —asignar recursos a los eventos según algún criterio—, para la que el catálogo menciona *first-in/first-out* y *round robin* como ejemplos. Se eligió `least_conn` sobre *round-robin* porque las peticiones tienen costos heterogéneos (`/api/report?iter=...` es mucho más cara que `/whoami`) y *round-robin* repartiría por cantidad, no por ocupación real.
4. **Reintento transparente** (`proxy_next_upstream`) y detección de réplicas caídas (`max_fails=2` `fail_timeout=5s`).

Por qué satisface los requerimientos.

- **RNF-01.** El trabajo de `/api/report` es CPU-bound e indivisible: una sola instancia con 0,5 vCPU lo procesa esencialmente en serie — es exactamente la contención que la táctica busca eliminar. Con tres copias, tres peticiones avanzan simultáneamente en núcleos distintos, y el tiempo del lote se divide aproximadamente por el número de copias. La medición del script `demo-rendimiento.sh` da una **aceleración de ~3,1x** (14,6 s → 4,6 s para el mismo lote de 60 peticiones), con la carga repartida en ~34 % / 33 % / 33 %.
- **RNF-02.** Agregar capacidad es declarar un servicio más en `docker-compose.yaml` y una línea más en el `upstream` de nginx. Los clientes siguen apuntando a la misma URL porque el balanceador es el único punto de entrada: la topología interna les resulta invisible.
- **RNF-03.** Al existir copias redundantes, la caída de una no interrumpe el servicio: el balanceador la marca como no disponible y reencamina. La táctica fue elegida por rendimiento, pero la redundancia que introduce **también** aporta disponibilidad: funciona como una **replicación** —copias exactas de un componente listas para tomar su lugar si una falla—, que es una táctica de redundancia del catálogo de disponibilidad reutilizada acá sin costo adicional.

Limitación honesta. La táctica mejora el *throughput* agregado, no la latencia de una petición individual: una petición aislada tarda lo mismo con 1 o con 3 réplicas. Además, el beneficio se sostiene solo mientras haya CPU física disponible; más réplicas que núcleos no aceleran nada.

T2 — Autenticar actores → RNF-04

En qué consiste la táctica. Dentro de la rama *resistir ataques*, autenticar actores significa **verificar que un actor es quien dice ser** antes de concederle acceso a cualquier recurso. El catálogo enumera como mecanismos las contraseñas, los *one-time passwords*, los certificados digitales, el doble factor y la identificación biométrica; acá se usa el primero, reforzado por un token firmado. Es la táctica que sigue naturalmente a **identificar actores** —conocer la identidad de quien interactúa con el sistema, típicamente por *user id*— y precede a **autorizar actores** (RNF-07).

Cómo se implementó.

1. **Verificación de credenciales.** Las contraseñas no se almacenan: se guarda `script(password, salt)` con un *salt* aleatorio por usuario. `script` es una función deliberadamente costosa en CPU y memoria, lo que encarece un ataque de diccionario sobre el almacén aunque este se filtre.
2. **Comparación en tiempo constante** (`crypto.timingSafeEqual`), para que el tiempo de respuesta no revele cuántos bytes del hash coincidieron.
3. **Hash señuelo.** Si el usuario no existe, igualmente se ejecuta un `script` contra un registro falso. Así el tiempo de respuesta es indistinguible y un atacante no puede **enumerar qué cuentas existen** midiendo latencias.
4. **Emisión de un JWT HS256** firmado con clave secreta, con `sub` (sujeto), `rol`, `iss` (emisor) y `exp` (expiración a 15 minutos).
5. **Verificación en cada petición protegida:** se recalcula el HMAC sobre `header.payload` y se compara en tiempo constante; se validan además el emisor y la expiración. Cualquier alteración de un solo bit del token invalida la firma.

Por qué satisface RNF-04. Ningún recurso bajo `/api/` ejecuta lógica de negocio antes de que el token sea validado: la autenticación es una barrera previa, no una comprobación intercalada que pueda olvidarse en un endpoint nuevo. La firma HMAC hace **computacionalmente inviable** falsificar un token sin la clave, y la expiración acota la ventana de uso de un token robado. La demostración verifica los cuatro casos: sin token (401), credenciales inválidas (401), token adulterado (401, “firma inválida”), token de un emisor no confiable (401), y token legítimo (200).

Por qué la táctica encaja con T1. Al ser el token autocontenido y verificable localmente, la autenticación **no introduce estado compartido ni un cuello de botella**: cada réplica valida por su cuenta. Una alternativa basada en sesiones en memoria habría roto la intercambiabilidad de las copias y, por lo tanto, T1.

T3 — Limitar el acceso → RNF-05, RNF-06, RNF-07

En qué consiste la táctica. También en *resistir ataques*, limitar el acceso **reduce la superficie de ataque restringiendo la cantidad de puntos de acceso o el tipo de tráfico permitido** — el ejemplo canónico del catálogo es una DMZ; acá, el gateway como único punto de entrada cumple ese papel. En la práctica se decide qué está expuesto, desde dónde, con qué verbos y a qué ritmo. Es complementaria a la autenticación: mientras T2 pregunta “¿quién sos?”, T3 limita “¿por dónde y cuánto podés golpear?”. Se le suman dos tácticas vecinas de la misma rama: **limitar la exposición** (minimizar la información o los servicios que revela cada punto de acceso) y **separar entidades** (aislar componentes para que el compromiso de uno no alcance a los demás, que el catálogo ejemplifica con máquinas físicas, VMs y contenedores).

Cómo se implementó (cinco controles en tres capas).

Red (Docker)	Réplicas y Redis en una red <code>internal: true</code> , sin puertos publicados	Solo existe un punto de entrada al sistema
Borde (nginx)	<code>limit_req</code> por IP: 200 r/s general, 60 r/min en <code>/auth/login</code> ; <code>limit_conn</code>	El ritmo de interacción, absorbido antes de llegar a la aplicación
Borde (nginx)	Lista blanca de verbos (<code>GET/POST/HEAD/OPTIONS</code>), <code>client_max_body_size 64k</code> , <code>server_tokens off</code>	El contrato admitido y la información revelada
Aplicación	Bloqueo de cuenta: 5 fallos en 60 s → 60 s de bloqueo (táctica <i>restringir el ingreso</i>)	Los intentos por cuenta , no solo por IP
Aplicación	Control por rol en operaciones de escritura (táctica <i>autorizar actores</i>)	El alcance de un actor ya autenticado
Contenedor	<code>read_only</code> , <code>cap_drop: ALL</code> , <code>no-new-privileges</code> , usuario no-root (táctica <i>separar entidades</i>)	Lo que un atacante podría hacer tras comprometer un proceso

Por qué satisface RNF-05. Los dos frenos son deliberadamente independientes y atacan dimensiones distintas del problema:

- El **rate limit del gateway es por IP** y protege a *todo* el sistema, incluso de peticiones que ni siquiera son intentos de login válidos. Su virtud es que el tráfico abusivo **se descarta en el borde**, sin consumir CPU de las réplicas ni ejecutar script (que es caro por diseño); de lo contrario, el propio mecanismo de T2 se convertiría en un vector de denegación de servicio.
- El **bloqueo de cuenta es por usuario** —la táctica *restringir el ingreso*, cuyo ejemplo en el catálogo es literalmente “bloquear a un usuario tras varios intentos fallidos de autenticación” — y sigue funcionando aunque el atacante distribuya el ataque entre muchas IPs, escenario en el que el rate limit por IP sería inútil.

Ninguno de los dos alcanza por separado; juntos cubren tanto el ataque concentrado como el distribuido. La demostración los distingue explícitamente: las respuestas generadas por una réplica llevan la cabecera `X-Served-By`, las cortadas por el gateway no.

Por qué satisface RNF-06. Un atacante en la red del host solo encuentra el puerto del gateway; los puertos 3000 y 6379 no responden. Aunque comprometiera una réplica, la red `internal` le impide iniciar conexiones hacia afuera, lo que dificulta la exfiltración y el establecimiento de un canal de control.

Por qué satisface RNF-07. El control de rol es la táctica de **autorizar actores**: determinar qué acciones o recursos puede acceder un actor ya autenticado, limitándolo por rol. Se aplica después de autenticar, sobre los *claims* firmados del token: un usuario no puede escalar privilegios manipulando el `rol` del JWT porque eso invalidaría la firma. Autenticación y autorización se apoyan una en la otra.

4. Interacción entre las tácticas (trade-offs)

Las tácticas no son independientes; documentar cómo se afectan entre sí es parte del diseño:

- **T2 sobre T1 (positiva por diseño).** La autenticación con token autocontenido se eligió *porque* preserva la ausencia de estado que T1 necesita. Sesiones en memoria habrían obligado a *sticky sessions*, degradando el reparto de carga.
- **T3 sobre T1 (tensión real).** Vista desde el catálogo de rendimiento, la limitación de tasa del gateway es una táctica de la rama opuesta a T1: **gestionar la llegada de eventos**, cuyo ejemplo es “un SLA que limita la cantidad de eventos por unidad de tiempo”. Se está controlando la demanda para resistir un ataque, y eso es un límite deliberado al rendimiento. Se resolvió con **zonas separadas**: `/auth/login` con un límite estricto (60 r/min) y el tráfico general con uno holgado (200 r/s), de modo que el control de seguridad no estrangule la ruta que RNF-01 debe cumplir. Este es el trade-off central del trabajo y la razón de que las zonas no sean una sola.
- **T2 sobre el rendimiento (costo asumido).** `script` es caro a propósito. Es un costo aceptable porque se paga solo en el login, no en cada petición; el resto de las peticiones solo verifican un HMAC, que es barato.
- **T3 protegiendo a T2.** Sin el límite del borde, el costo de `script` convertiría a `/auth/login` en un amplificador de denegación de servicio.
- **Sobre la eficiencia energética.** El catálogo define “recurso” como un dispositivo computacional que consume energía al proveer su funcionalidad, y su rama de *reducir la demanda de recursos* reutiliza las mismas tácticas de rendimiento. Descartar el tráfico abusivo en el borde, por lo tanto, no solo protege a las réplicas: evita trabajo —y consumo— que de otro modo se pagaría en cada instancia.

5. Verificación

La Parte 2 incluye scripts que producen la evidencia de cada requerimiento:

RNF-01	Comparación del mismo lote con 3 réplicas y con 1	<code>scripts/demo-rendimiento.sh</code> pasos 3, 5 y 6
RNF-02	Reparto observable + inspección de la configuración	<code>scripts/demo-rendimiento.sh</code> pasos 2 y 4
RNF-03	Se detienen 2 réplicas y se mide el error rate	<code>scripts/demo-rendimiento.sh</code> paso 7
RNF-04	Sin token, token adulterado, token ajeno, token válido	<code>scripts/demo-seguridad.sh</code> A.1–A.6
RNF-05	Bloqueo de cuenta y rate limit, medidos por separado	<code>scripts/demo-seguridad.sh</code> B.3 y B.4
RNF-06	Puertos publicados, acceso directo a 3000/6379, verbos	<code>scripts/demo-seguridad.sh</code> B.5–B.7
RNF-07	Misma operación con rol <code>user</code> y con rol <code>admin</code>	<code>scripts/demo-seguridad.sh</code> B.1 y B.2

Resultados obtenidos en la ejecución de referencia

```

RENDIMIENTO
1 replica : 14.61 s ( 4.11 req/s)

```

```
3 replicas : 4.64 s ( 12.94 req/s)
Aceleracion : x3.15 (60 peticiones CPU-bound, concurrencia 12)
```

```
Reparto de carga: api-1 34.4 % | api-2 32.8 % | api-3 32.8 %
Con 2 de 3 replicas caidas: 6/6 peticiones respondidas con HTTP 200
```

SEGURIDAD

```
Sin token / token adulterado / token ajeno ..... 401 (3/3 rechazados)
Token valido ..... 200
Escritura con rol 'user' ..... 403
Escritura con rol 'admin' ..... 201
Bloqueo de cuenta ..... 5 x 401, luego 429 "cuenta bloqueada"
Rate limit del gateway ..... 11 x 200, 39 x 429 cortadas en el borde
Puertos 3000 y 6379 desde el host ..... sin respuesta
Verbo DELETE ..... 405
```

6. Fuentes

El vocabulario, la clasificación en ramas y las definiciones de cada táctica de este documento provienen del material teórico de la Unidad 2 de la asignatura:

12	Tácticas para el rendimiento	https://ferqueve.github.io/clases/ada2/clase12/index.html
13	Tácticas para la protección	https://ferqueve.github.io/clases/ada2/clase13/index.html
14	Tácticas para la seguridad	https://ferqueve.github.io/clases/ada2/clase14/index.html
15	Tácticas para la facilidad de modificación	https://ferqueve.github.io/clases/ada2/clase15/index.html
16	Tácticas para la facilidad de despliegue	https://ferqueve.github.io/clases/ada2/clase16/index.html
17	Tácticas para la eficiencia energética	https://ferqueve.github.io/clases/ada2/clase17/index.html